Switching Costs, Solvers, and Sensitivity

BUS 104: Decision Analysis and Management Science

Danko Turcic

October 27, 2026

UC RIVERSIDE | UNIVERSITY OF CALIFORNIA | School of Business
A. GARY ANDERSON GRADUATE SCHOOL OF MANAGEMENT

Study Resource: BUS 104 NotebookLM (AI Course Companion)

Official BUS 104 Class Notes & Midterm Study Assistant

Click Here: Open BUS 104 Class Notes in Google NotebookLM

`https://notebook.google.com/notebook/21540e36-100c-4abc-a851-af3bf66ef8e9`

What Is Inside?

- **Complete Notes:** All BUS 104 slides & class notes.
- **Solver Guides:** LP, MILP, `PuLP 4.0`, & `OpenSolver`.
- **Zero Hallucination:** Grounded strictly in our course files.

How to Use It for the Midterm

- **Concept Recaps:** Ask about Big-M, `c.pi`, or `v.dj`.
- **Self-Testing:** Generate practice quizzes & flashcards.
- **Code Check:** Query `PuLP` & Google Sheets syntax.

Lecture Roadmap: Formulate → Code → Sensitivity

Case study: **Suncrest Artisan Lighting** across two main parts:

1. Formulate Switching-Cost MILP (Slides 4–9)

- **Setup Jump:** Why fixed costs break LP.
- **Binary Switches:** $y_i \in \{0, 1\}$ and Big-M links.
- **Baseline Optimum:** $(8, 0)$ vs. $(6, 4)$.

2. Code MILP in Python & Sheets (Slides 10–21)

- **Python `PuLP 4.0`:** Live cell-by-cell build.
- **Google Sheets:** `OpenSolver` grid setup.
- **Live Scenario:** Cutting setup cost F_2.

3. Slack & Shadow Price Basics (Slides 22–27)

- **Drop Switches:** Solve 2-product LP in Python.
- **Slack (`c.slack`):** Leftover hours.
- **Shadow Price (`c.pi`):** Value of $+1$ hour.

4. Reduced Cost Basics (Slides 28–31)

- **Definition (`v.dj`):** Profit minus resource cost.
- **Python Test:** Adding a 3rd product (x_3) at \$25 vs. \$35.

The Business Case: Suncrest Artisan Lighting

Products: Desk Lamps (x_1) and **Floor Lamps (x_2)**. Any $x_i > 0$ triggers a **daily setup cost** F_i.

Department / Metric	Desk (x_1)	Floor (x_2)	Daily Limit
Assembly (hrs/unit)	1 hr	1 hr	≤ 10 hrs
Machining (hrs/unit)	2 hrs	1 hr	≤ 16 hrs
Finishing (hrs/unit)	0 hrs	1 hr	≤ 7 hrs
Gross Margin (c_i)	**\$50**/unit	**\$30**/unit	—
Switching Cost (F_i)	**\$80** if $x_1 > 0$	**\$60** if $x_2 > 0$	—

Core Managerial Trade-Off

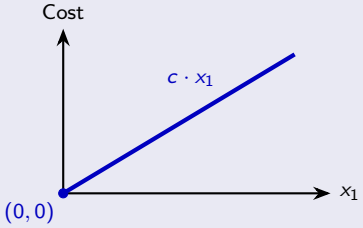
- **Diversify ($x_1 > 0, x_2 > 0$):** Higher gross margin, pay **both setups** (\$140).
- **Specialize ($x_1 > 0, x_2 = 0$):** Idle Finishing hours, **save** \$60 **setup**!

Why Switching Costs Break Standard Linear Programming

Standard LP requires proportional cost ($C(x_i) = k \cdot x_i$). Setup costs **jump at** $x_i = 0$:

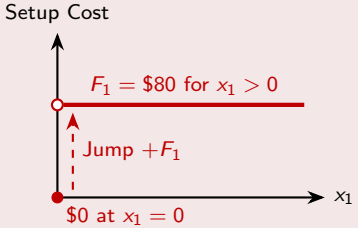
$$\text{Setup Cost}_i(x_i) = \begin{cases} \$0 & \text{if } x_i = 0 \quad \text{(Line } i \text{ OFF)} \\ F_i & \text{if } x_i > 0 \quad \text{(Line } i \text{ ON)} \end{cases}$$

Graph A: Standard LP (Proportional)



Smooth through $(0, 0)$: Pure LP works.

Graph B: Fixed Switching Cost (MILP)



Jump at 0 requires **binary switch** $y_i \in \{0, 1\}$.

Step 1: Decision Variables & Net Profit Objective

Formulate as a **Mixed-Integer Linear Program (MILP)** with **four variables**:

1. Continuous Production ($x_1, x_2 \geq 0$)

- $x_1 \geq 0$: **Desk Lamps** produced.
- $x_2 \geq 0$: **Floor Lamps** produced.

2. Binary Switches ($y_1, y_2 \in \{0, 1\}$)

- $y_1 = 1$ if Desk line ON (0 if OFF).
- $y_2 = 1$ if Floor line ON (0 if OFF).

Net Profit Objective Function

$$\max Z = \underbrace{50x_1 + 30x_2}_{\text{Gross Margin}} - \underbrace{(80y_1 + 60y_2)}_{\text{Switching Costs}}$$

The "Solver Cheating" Trap

- $-80y_1 - 60y_2$ penalizes profit Z.
- Without links, solver sets $\mathbf{y_i = 0}$ while $\mathbf{x_i > 0}$!

Step 2: Big-M Linking Constraints (Two Angles)

Enforce the logical rule: **"If $x_i > 0$, then $y_i = 1$"** via linear inequalities:

Angle 1: Mathematical Gate / Valve

$$x_i \leq M_i y_i \quad \Longleftrightarrow \quad \mathbf{x_i - M_i y_i \leq 0}$$

- **Switch OFF ($y_i = 0$):** $x_i \leq 0 \implies \mathbf{x_i = 0}$ (Gate shut).
- **Switch ON ($y_i = 1$):** $x_i \leq M_i$ (Gate open up to M_i).

Angle 2: Tightest Physical Upper Bound M_i

Find max feasible x_i from department limits:

- **Desk (x_1):** Assembly ≤ 10, Machining $2x_1 \leq 16 \implies x_1 \leq 8$:

$$M_1 = \min\{10, 8\} = \mathbf{8} \implies \mathbf{x_1 - 8y_1 \leq 0}$$

- **Floor (x_2):** Assembly ≤ 10, Machining ≤ 16, Finishing ≤ 7:

$$M_2 = \min\{10, 16, 7\} = \mathbf{7} \implies \mathbf{x_2 - 7y_2 \leq 0}$$

Step 3: Complete MILP Formulation in Standard Form

Variables on Left-Hand Side (LHS), constants on Right-Hand Side (RHS):

Algebraic MILP Model

$$\begin{aligned}
\max \quad & Z = 50x_1 + 30x_2 - 80y_1 - 60y_2 \\
\text{s.t.} \quad & x_1 + x_2 \leq 10 && \text{(Assembly hrs)} \\
& 2x_1 + x_2 \leq 16 && \text{(Machining hrs)} \\
& x_2 \leq 7 && \text{(Finishing hrs)} \\
& \mathbf{x_1 - 8y_1 \leq 0} && \textbf{(Desk Link, } M_1 = 8\textbf{)} \\
& \mathbf{x_2 - 7y_2 \leq 0} && \textbf{(Floor Link, } M_2 = 7\textbf{)} \\
& x_1, x_2 \geq 0, \quad y_1, y_2 \in \{0, 1\}
\end{aligned}$$

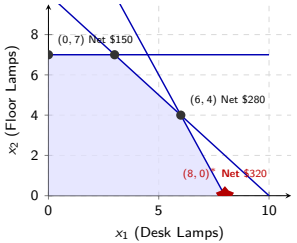
Why Standard Form $x_i - M_i y_i \leq 0$?

- **Python (`PuLP`):** Accepts either `x1 - 8*y1 <= 0` or `x1 <= 8*y1`.
- **Google Sheets:** Put 1 under x_1 and -8 under y_1 with RHS $= 0$.
- **One Formula:** Drag `=SUMPRODUCT()` down all 5 constraint rows!

Step 4: Tracing All 4 Switch Combinations by Hand & Graph

Evaluate the best corner for each of the $2^2 = 4$ switch regimes (y_1, y_2):

Switches (y_1, y_2)	**Best** (x_1, x_2)	**Gross** $50x_1 + 30x_2$	**Setup** $80y_1 + 60y_2$	**Net Profit** Z
$(0,0)$	$(0,0)$	\$0	\$0	\$0
$(0,1)$	$(0,7)$	\$210	\$60	\$150
$(1,1)$	$(6,4)$	\$420	\$140	\$280
$(\mathbf{1,0})^*$	$(\mathbf{8,0})^*$	**\$400**	**\$80**	**\$320***



Why $(8,0)$ Beats $(6,4)$

- $(8,0) \to (6,4)$ gains $+$**\$20** gross margin.
- But triggers $+$**\$60** Floor setup $\implies$ $-$**\$40** net!

Part 2: 4-Step MILP Blueprint in Python (PuLP 4.0)

Let's immediately code our MILP in PuLP 4.0 using four compact steps:

Step 1: Initialize Model

```
prob = pl.LpProblem("Name", pl.LpMaximize)
```

Step 2: Define Variables

- Quantities x_i: `cat="Continuous"`
- Switches y_i: `cat="Binary"`

Step 3: Add Objective & Constraints

- Objective: `prob += expr, "Name"`
- Constraints: `prob += lhs <= rhs, "Name"`

Step 4: Solve & Print

- Solve: `res = prob.solve()`
- Inspect: `pl.value(prob.objective)`, `v.varValue`

PuLP Steps 1 & 2: Model & Decision Variables

Initialize the maximization model and declare continuous (x_1, x_2) and binary (y_1, y_2) variables:

```
import pulp as pl

# Step 1: Initialize the MILP model
prob = pl.LpProblem("Suncrest_MILP", pl.LpMaximize)

# Step 2a: Continuous production quantities (x1, x2 >= 0)
x1 = prob.add_variable("x1", lowBound=0, cat="Continuous")
x2 = prob.add_variable("x2", lowBound=0, cat="Continuous")

# Step 2b: Binary setup switches (y1, y2 in {0, 1})
y1 = prob.add_variable("y1", cat="Binary")
y2 = prob.add_variable("y2", cat="Binary")
```

Key Syntax Points

- `cat="Binary"`: Automatically enforces $y_i \in \{0, 1\}$.
- `lowBound=0`: Enforces non-negativity $x_1, x_2 \geq 0$.

PuLP Step 3a: Objective & Department Constraints

Use `prob += ...` to append the net profit objective and three department capacities:

```
# Step 3a: Net Profit Objective (Gross Margin minus Switching Costs)
prob += 50 * x1 + 30 * x2 - 80 * y1 - 60 * y2, "Net_Profit"

# Step 3b: Department Capacity Constraints
prob += 1 * x1 + 1 * x2 <= 10, "Assembly"
prob += 2 * x1 + 1 * x2 <= 16, "Machining"
prob += 1 * x2 <= 7, "Finishing"
```

Why Label Constraints?

- **Easy Verification:** Shows up cleanly in `print(prob)`.
- **Sensitivity Loops:** Identifies each row via `c.name`.

PuLP Step 3b: Big-M Linking Constraints

Append the two Big-M linking constraints ($M_1 = 8, M_2 = 7$):

```
# Step 3c: Big-M Linking Constraints (Standard Form: x_i - M_i * y_i <= 0)
prob += x1 - 8 * y1 <= 0, "Link_Desk"
prob += x2 - 7 * y2 <= 0, "Link_Floor"
```

Alternative Form in PuLP

```
prob += x1 <= 8 * y1, "Link_Desk"
prob += x2 <= 7 * y2, "Link_Floor"
```

Both forms produce the exact same model.

If Omitted ($\rightarrow$ Cheating)

- Solver sets `y1 = 0, y2 = 0` with `x1 = 6, x2 = 4`.
- Falsely reports $420 profit (skips setups!).

PuLP Step 4: Solving & Simple Print Statements

Call `prob.solve()` and print status, objective value, and each variable:

Python Code (Step 4):

```
# Step 4: Solve the MILP and print results
res = prob.solve()

print("Status:", res.status.name)
print("Optimal Profit =", pl.value(prob.objective))
print("x1 =", x1.varValue)
print("x2 =", x2.varValue)
print("y1 =", y1.varValue)
print("y2 =", y2.varValue)
```

Console Output:

```
Status: Optimal
Optimal Profit = 320.0
x1 = 8.0
x2 = 0.0
y1 = 1.0
y2 = 0.0
```

Optimal MILP Plan

- **Line 1 ON (`y1=1`):** `x1 = 8.0` Desk Lamps.
- **Line 2 OFF (`y2=0`):** `x2 = 0.0`, Net **$320.0**.

Complete MILP Script & Variable Loop

Full 16-line script using a 2-line `for` loop over `prob.variables()`:

```
import pulp as pl

prob = pl.LpProblem("Suncrest_MILP", pl.LpMaximize)

x1 = prob.add_variable("x1", lowBound=0, cat="Continuous")
x2 = prob.add_variable("x2", lowBound=0, cat="Continuous")
y1 = prob.add_variable("y1", cat="Binary")
y2 = prob.add_variable("y2", cat="Binary")

prob += 50*x1 + 30*x2 - 80*y1 - 60*y2, "Net_Profit"
prob += 1*x1 + 1*x2 <= 10, "Assembly"
prob += 2*x1 + 1*x2 <= 16, "Machining"
prob += 1*x2 <= 7, "Finishing"
prob += x1 - 8*y1 <= 0, "Link_Desk"
prob += x2 - 7*y2 <= 0, "Link_Floor"

res = prob.solve()
print("Status:", res.status.name)
print("Optimal Profit =", pl.value(prob.objective))
for v in prob.variables():
    print(v.name, "Value =", v.varValue)
```

Console Output:

```
Status: Optimal
Optimal Profit = 320.0
x1 Value = 8.0
x2 Value = 0.0
y1 Value = 1.0
y2 Value = 0.0
```

Debugging Tip: `print(prob)`

- Run `print(prob)` before `prob.solve()`.
- Displays full algebraic model to check signs!

Google Sheets Grid Layout (OpenSolver)

Place all four decision variables (x_1, x_2, y_1, y_2) in **Row 4 (B4:E4)**:

	A	B	C	D	E	F	G	H
3	**Variable**	**Desk x_1**	**Floor x_2**	**Switch y_1**	**Switch y_2**	**LHS Formula**	**Sign**	**RHS**
4	**Decision Vars**	**8**	**0**	**1**	**0**			
5	**Unit Profit**	50	30	−80	−60	**320**	=SUMPRODUCT(B4:E4,B5:E5)	
6	**Constraints**					**Used (LHS)**		**Limit**
7	Assembly Hrs	1	1	0	0	8	$\leq$	10
8	Machining Hrs	2	1	0	0	16	$\leq$	16
9	Finishing Hrs	0	1	0	0	0	$\leq$	7
10	Link Desk ($M_1 = 8$)	1	0	−8	0	0	$\leq$	0
11	Link Floor ($M_2 = 7$)	0	1	0	−7	0	$\leq$	0

- **Yellow (B4:E4):** Variable cells. **Green (F5):** Objective =SUMPRODUCT(B4:E4, B5:E5).
- **Blue (F7:F11):** LHS formula =SUMPRODUCT($B$4:$E$4, B7:E7) dragged down!

Why Standard Form ($x_1 - 8y_1 \leq 0$) Simplifies Sheets

Two valid ways to enter Rows 10–11 (**Big-M Links**) in Google Sheets:

Method A: Standard Form (**Best**)

Write $\mathbf{1} \cdot \mathbf{x_1} + \mathbf{0} \cdot \mathbf{x_2} - \mathbf{8} \cdot \mathbf{y_1} + \mathbf{0} \cdot \mathbf{y_2} \leq \mathbf{0}$:

- **Cells:** `B10 = 1`, `D10 = -8`, `H10 = 0`.
- **Formula in** `F10`:

```
=SUMPRODUCT($B$4:$E$4, B10:E10)
```

- **One Block:** Enter `F7:F11 <= H7:H11` in OpenSolver!

Method B: Dynamic RHS Cell

Keep $x_1 \leq 8y_1$ directly:

- **LHS Cell** `F10`: `=B4`
- **RHS Cell** `H10`: `=8*D4`
- **Rule:** Always put `=8*D4` in a **sheet cell**, never inside the OpenSolver dialog!

Step-by-Step OpenSolver Sidebar Setup

Open **Extensions** → **OpenSolver** → **Open Sidebar**:

OpenSolver Settings

1. **Objective Cell:** `F5` (→ **Max**)
2. **Variable Cells:** `B4:E4`
3. **Constraints:**
   - `F7:F11 <= H7:H11` (All 5 rows)
   - `D4:E4 bin` (Binary switches y_1, y_2)
4. **Non-Negative:** Check ☒ ($x_1, x_2 \geq 0$)
5. **Engine:** CBC → Click **Solve Model**

Critical Check: Only `D4:E4` Are `bin`!

- **Never** set `B4:E4` as `bin` (forces $x_1, x_2 \in \{0, 1\}$!).
- **`B4:C4` (x_1, x_2):** Continuous.
- **`D4:E4` (y_1, y_2):** Binary (`bin`).

Three Common Midterm Pitfalls to Avoid

Quick checklist before running any MILP scenario:

Pitfall 1: Forgetting `bin` / `cat="Binary"` on y_1, y_2

Result: Solver picks fractional switches ($y_1 = x_1/8$, $y_2 = x_2/7$) and underpays setup costs!

Pitfall 2: Choosing M_i Too Small (Artificially Clipping Output)

Result: Setting $M_1 = 5$ ($x_1 - 5y_1 \leq 0$) caps Desk Lamps at 5 and cuts off the true optimum $(8, 0)$!

Pitfall 3: Missing Dollar Signs `$B$4:$E$4` in `SUMPRODUCT`

Result: Dragging `=SUMPRODUCT(B4:E4, B7:E7)` shifts variable cells down to Row 5, Row 6, etc.!

Interactive Exercise: You Make the Call #1 (Live MILP Scenario)

Scenario: Quick-Change Tooling on Line 2

Engineering cuts the Floor Lamp setup cost from $F_2 = \$60$ down to $\mathbf{F_2^{new} = \$15}$ (Desk setup stays $F_1 = \$80$). Test this by hand **or** by changing `- 60*y2` to `- 15*y2` in Python / Cell E5 in Sheets!

Answer all three questions (2 minutes):

1. **New Net Profit:** What is Z at $(x_1, x_2) = (6, 4)$ with $(y_1, y_2) = (1, 1)$?
2. **Optimal Plan:** Open both lines at $(6, 4)$ or stay Desk-only at $(8, 0)$ (Net \$320)?
3. **Break-Even Setup F_2^*:** At what F_2^* does $(6, 4)$ tie $(8, 0)$?

FORMULA & PARAMETER REFERENCE: $\max Z = 50x_1 + 30x_2 - 80y_1 - F_2y_2$ | **At** $(8, 0), (1, 0)$**:** Gross $= \$400$, Net $= 400 - 80 = \mathbf{\$320}$ | **At** $(6, 4), (1, 1)$**:** Gross $= 50(6) + 30(4) = \mathbf{\$420}$

Removing Switching Costs: Back to a Simple 2-Product LP

What if both machines are **already set up** ($F_1 = 0, F_2 = 0$)? Drop y_1 and y_2:

The 2-Product Continuous LP

$$\begin{aligned} \max \quad & Z = 50x_1 + 30x_2 \\ \text{s.t.} \quad & 1x_1 + 1x_2 \leq 10 \quad \text{(Assembly)} \\ & 2x_1 + 1x_2 \leq 16 \quad \text{(Machining)} \\ & 1x_2 \leq 7 \quad \text{(Finishing)} \\ & x_1, x_2 \geq 0 \end{aligned}$$

Optimal LP Solution

- $\mathbf{x_1^* = 6}$ Desk Lamps
- $\mathbf{x_2^* = 4}$ Floor Lamps
- **Optimal Profit:**

$$Z^* = 50(6) + 30(4) = \$\mathbf{420}$$

*Why drop binary switches? Because **Shadow Prices** and **Reduced Costs** only exist for continuous LPs.*

Solving the 2-Product LP in Python (PuLP)

Let's solve this 2-product LP in Python and print `c.pi` (Shadow Price) and `c.slack` (Slack):

Python Code (Matches Notebook Part 2):

```
import pulp as pl

lp = pl.LpProblem("Suncrest_LP", pl.LpMaximize)

x1 = lp.add_variable("x1", lowBound=0, cat="Continuous")
x2 = lp.add_variable("x2", lowBound=0, cat="Continuous")

lp += 50 * x1 + 30 * x2, "Profit"
lp += 1 * x1 + 1 * x2 <= 10, "Assembly"
lp += 2 * x1 + 1 * x2 <= 16, "Machining"
lp += 1 * x2 <= 7, "Finishing"

res = lp.solve()
print("Profit =", pl.value(lp.objective))

for c in lp.constraints():
    print(c.name, "Shadow Price =", c.pi, "Slack =", c.slack)
```

Console Output:

```
Profit = 420.0
Assembly Shadow Price = -10.0 Slack = -0.0
Machining Shadow Price = -20.0 Slack = -0.0
Finishing Shadow Price = 0.0 Slack = 3.0
```

How to Read This Output

- `-0.0` just means **0**.
- Shadow Prices in magnitude are **\$10**, **\$20**, and **\$0**.

Definition 1: What Is Slack? (`c.slack`)

Definition: Slack (`c.slack`)

Slack = (Hours Available) − (Hours Used at Optimum)

At our optimal solution $\mathbf{x_1 = 6}$, $\mathbf{x_2 = 4}$:

Department	Hours Used	Hours Available	Slack (`c.slack`)	Status
Assembly	$1(6) + 1(4) = 10$	10	**0**	**Binding** (100% full)
Machining	$2(6) + 1(4) = 16$	16	**0**	**Binding** (100% full)
Finishing	$0(6) + 1(4) = 4$	7	**3**	**Non-Binding** (3 hrs leftover)

Definition 2: What Is a Shadow Price? (c.pi)

Definition: Shadow Price (c.pi)

Shadow Price = How much **Optimal Profit (Z^*)** goes up if we get **+1 more hour** of that resource.

Our 3 Shadow Prices Explained

- **Assembly Shadow Price** = \$10**:** +1 extra hour of Assembly increases profit by **\$10**.
- **Machining Shadow Price** = \$20**:** +1 extra hour of Machining increases profit by **\$20**.
- **Finishing Shadow Price** = \$0**:** +1 extra hour of Finishing increases profit by **\$0**.

Key Rule

If a resource already has leftover hours (Slack > 0), its Shadow Price is **always \$0**!

How Is a Shadow Price Computed? (+1 Hour Test in Python)

Why is Machining's Shadow Price **$20**? Give Machining +1 **extra hour** (16 → 17) and re-solve:

Change 16 to 17 in Python:

```
# Change Machining limit from 16 to 17
lp += 1 * x1 + 1 * x2 <= 10, "Assembly"
lp += 2 * x1 + 1 * x2 <= 17, "Machining"
lp += 1 * x2 <= 7, "Finishing"

lp.solve()
print("New Profit =", pl.value(lp.objective))
print("x1 =", x1.varValue, "x2 =", x2.varValue)
```

Result

- **Old Profit (16 hrs): $420** (at $x_1 = 6, x_2 = 4$)
- **New Profit (17 hrs): $440** (at $x_1 = 7, x_2 = 3$)
- **Profit Increase:**

$$\$440 - \$420 = +\mathbf{\$20}$$

How to Use Shadow Prices: Pricing a New Product

Suppose We Consider a 3rd Product: Pendant Lamp (x_3)

- Sells for **$25 profit** per lamp.
- Uses **1 hr Assembly**, **1 hr Machining**, and **1 hr Finishing**.

Step 1: Value of Resources Used by 1 Pendant

- **Assembly:** 1 hr $\times$ \$10 = \$10
- **Machining:** 1 hr $\times$ \$20 = \$20
- **Finishing:** 1 hr $\times$ \$0 = \$0
- **Total Resource Value Used** = **\$30**

Step 2: Should We Make It?

- **NO!**
- It uses **$30** worth of resources to earn only **$25** profit.
- We lose **$5** on every Pendant Lamp made!

Definition 3: What Is Reduced Cost? (v.dj)

Definition: Reduced Cost (v.dj)

Reduced Cost = (Unit Profit) − (Value of Resources Used)

For the **Pendant Lamp (x_3)**: Reduced Cost = \$25 − \$30 = −**\$5**.

Two Simple Meanings of This \$5 Shortfall

1. **Profit Penalty:** Forcing 1 unit of x_3 **reduces total profit by \$5** (from \$420 to \$415).
2. **Price Hike Needed:** We must **raise x_3's profit by at least \$5** (to \$30) before it is worth making!

*(Note: For products we are already making, $x_1 = 6$ and $x_2 = 4$, Reduced Cost is always **\$0**.)*

Computing Reduced Cost in Python: Adding x3 to PuLP

Add x3 (Pendant Lamp at $25 profit) to our Python LP and print `v.dj`:

Python Code (Adding x3 at $25 Profit):

```
lp3 = pl.LpProblem("Suncrest_3_Products", pl.LpMaximize)

x1 = lp3.add_variable("x1", lowBound=0, cat="Continuous")
x2 = lp3.add_variable("x2", lowBound=0, cat="Continuous")
x3 = lp3.add_variable("x3", lowBound=0, cat="Continuous")

lp3 += 50 * x1 + 30 * x2 + 25 * x3, "Profit"
lp3 += 1 * x1 + 1 * x2 + 1 * x3 <= 10, "Assembly"
lp3 += 2 * x1 + 1 * x2 + 1 * x3 <= 16, "Machining"
lp3 += 1 * x2 + 1 * x3 <= 7, "Finishing"

lp3.solve()
for v in lp3.variables():
    print(v.name, "Value =", v.varValue, "Reduced Cost =", v.dj)
```

Console Output:

```
x1 Value = 6.0 Reduced Cost = 0.0
x2 Value = 4.0 Reduced Cost = 0.0
x3 Value = 0.0 Reduced Cost = 5.0
```

Reading the Output for x3

- `x3 Value = 0.0`: Do not make any Pendant Lamps.
- `Reduced Cost = 5.0`: Profit is **$5 short** of the $30 resource cost!

Live Test in Python: Raising x3 Profit to $35

Since x3 needs more than $30 profit to be worth making, let's raise its profit from **$25 to $35** in Python:

Change `25 * x3` **to** `35 * x3`:

```
# Raise Pendant Lamp profit from $25 to $35
lp3 += 50 * x1 + 30 * x2 + 35 * x3, "Profit"

lp3.solve()
print("New Profit =", pl.value(lp3.objective))

for v in lp3.variables():
    print(v.name, "Value =", v.varValue, "Reduced Cost =", v.dj)
```

New Console Output:

```
New Profit = 440.0
x1 Value = 6.0 Reduced Cost = 0.0
x2 Value = 0.0 Reduced Cost = 5.0
x3 Value = 4.0 Reduced Cost = 0.0
```

Result

- Now we make $\mathbf{x_3 = 4}$ Pendant Lamps!
- x3 Reduced Cost becomes **0.0**, and profit rises to **$440**!

Summary Cheat Sheet: Sensitivity Analysis in PuLP

Everything you need to know about LP Sensitivity Analysis on one slide:

Concept	Simple Definition	Python Code	Key Rule
Slack	Leftover resource hours	`c.slack`	Slack $= 0 \implies$ **Binding** (full)
Shadow Price	Profit gain from $+1$ resource hour	`c.pi`	If Slack > 0, Shadow Price $=$ **\$0**
Reduced Cost	(Unit Profit) $-$ (Resource Value Used)	`v.dj`	Price hike needed if $x_j^* = 0$